

36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.

Appendix Overview

The appendix is organized into two parts: Appendix A–D provide related work and the main experimental setup of *RewardDS*, while Appendix E–J present additional experimental results to further demonstrate the effectiveness of *RewardDS*.

A Detailed Related Works

In this section, we provide a more detailed introduction to the related works on LLM privacy-preserving fine-tuning methods, including Anonymity-based methods, Encryption-based methods and Synthesis-based methods.

A.1 Anonymity-based methods

Anonymity-based methods aim to identify and remove user-specific private information from private data to enable privacy-preserving LLM fine-tuning. [Sweeney \(1997\)](#) achieves k-anonymity by dynamically identifying user-specific private information and applying substitution or removal to protect it. [Romanov et al. \(2019\)](#) employs a transformer framework with attention mechanisms to enhance anonymization performance. [Staab et al. \(2024\)](#) proposes an adversarial anonymization approach that leverages one LLM to anonymize user privacy while using another LLM to detect privacy information, iteratively improving the anonymization performance.

All of the above anonymity-based methods require detecting and removing user privacy, which will make the data incoherent and incomplete, thereby reducing its quality for downstream LLM fine-tuning.

A.2 Encryption-based methods

Encryption-based methods focus on applying encryption to the private data and maintaining secure communication between client and server to transmit the private data. [Lou et al. \(2020\)](#) applies fully Homomorphic encryption to protect private data, enabling privacy security while maintaining comparable model performance after fine-tuning on the encrypted data. [Frery et al. \(2025\)](#) combines the Low-Rank Adaptation technique and homomorphic encryption to improve the efficiency of LLM privacy-preserving fine-tuning. [You et al. \(2025\)](#) introduces the hybrid secret sharing algorithm by combining arithmetic secret sharing (ASS) and function secret sharing (FSS) to achieve secure

computation during LLM privacy-preserving fine-tuning.

However, current encryption-based methods still require numerous time and resources for encrypting private data and ensuring secure communication, making them impractical for real-world applications.

A.3 Synthesis-based methods

Synthesis-based methods have recently emerged as a more practical and reliable approach, which leverages synthetic data as a substitute for private data in LLM fine-tuning to balance utility and privacy.

[Kurakin et al. \(2023\)](#); [Yue et al. \(2023\)](#) propose using DP-SGD to locally fine-tune a lightweight model on the client as a generation proxy. This proxy model is used to generate synthetic data without privacy risks. The server utilizes the synthetic data to fine-tune the LLM, achieving privacy-preserving training. Considering that those synthetic data often contain numerous incoherent and flawed samples, [Yu et al. \(2024\)](#); [Hou et al. \(2024\)](#) filter out low-quality data by measuring similarity between synthetic and private data. Alternatively, [Wang et al. \(2024, 2022\)](#) avoid sampling synthetic data from the generation proxy model, instead using LLM on the server to improve the proxy model by distillation.

Nevertheless, only text similarity is too surface-level to accurately assess the quality of synthetic data for domain-specific tasks. Moreover, since the server-side LLM is not fine-tuned for domain-specific tasks, its ability to enhance the generation proxy model through distillation is limited.

B Details of Datasets

To evaluate the performance of the compared methods on domain-specific tasks, we focus on three tasks: Medical Question-Answering (QA), Financial QA, and Code Generation. For the medical QA task, we use the HealthCareMagic-100k dataset ([Li et al., 2023](#)); for the financial QA task, we use the fingpt-fiqq_qa dataset ([Zhang et al., 2023](#)); and for the code generation task, we use the opc-sft-stage2 dataset ([Huang et al., 2024](#)).

As [Dong et al. \(2024\)](#) points out, these public datasets suffer from a "data contamination" issue, where some of the data may have been used to train LLMs on the server, causing the models to memorize it and leading to unnaturally high performance. Moreover, the initial datasets are highly redundant,

containing many similar samples. To accurately assess the domain-specific performance of different baselines, we should pre-process these datasets. To be specific, firstly, we evaluate the dataset using the Qwen2.5-7B-Instruct model (Yang et al., 2024b) and exclude samples with high accuracy, as higher accuracy suggests these samples may have been part of the LLM’s training data and are thus contaminated.

After addressing the contamination issue, we use the Sentence-T5-Base model (Ni et al., 2022) to compute embeddings for each sample and calculate their similarity. This allows us to remove highly similar samples, ensuring deduplication. The pre-processed dataset is then split into private train set, dev set, and test set, with the detailed statistics shown in Table 7. For fair comparison across all methods, we control the size of our sampled synthetic dataset to be twice the size of the private training set, as shown in Table 7.

C Compared Methods

Here, we will provide more detailed introductions to all compared methods:

Vanilla LLM: Vanilla LLM directly uses the LLM (Qwen2.5-7B-Instruct) on the server for those domain-specific tasks.

Locally Fine-tuning: Locally Fine-tuning fine-tunes a lightweight model (Qwen2.5-0.5B-Instruct) using the private data on the client for those domain-specific tasks.

DP-Generation: As proposed by Kurakin et al. (2023), DP-Generation first uses DP to fine-tune a lightweight model (Qwen2.5-0.5B-Instruct) as the Generation Proxy Model on the client side. Then, it transmits the Generation Proxy Model to the server for synthetic data sampling. Then, the synthetic data is used to fine-tune the LLM (Qwen2.5-7B-Instruct) on the server for those domain-specific tasks.

DP-Instruct: Compared to DP-Generation, DP-Instruct (Yu et al., 2024) introduces a filtering step to improve the quality of synthetic data. After sampling synthetic data from the Generation Proxy Model, it computes the text similarity between the synthetic data and those private data. It filters out those low-similar data to improve data quality. Then the filtered synthetic data is used to fine-tune the LLM (Qwen2.5-7B-Instruct) on the server for those domain-specific tasks.

KnowledgeSG: Considering the high noise in synthetic data, KnowledgeSG Wang et al. (2024) avoids directly sampling synthetic data from the Generation Proxy Model. Instead, it distills knowledge from the LLM to enhance the Generation Proxy Model for domain-specific tasks. Specifically, KnowledgeSG first uses DP to fine-tune a lightweight model (Qwen2.5-0.5B-Instruct) as the Generation Proxy Model on the client. Then, it transmits the Generation Proxy Model to the server and generates synthetic instructions. The synthetic instructions are fed into the professional LLM (Qwen2.5-7B-Instruct) to generate high quality responses. By using the high quality responses to fine-tune the Generation Proxy Model, it can distill the capacity of LLM into the Generation Proxy Model. Finally, the Generation Proxy Model serves for those domain-specific tasks.

D Implementation Details

We use the Qwen2.5-0.5B-Instruct (Yang et al., 2024b) as the backbone for both the generation proxy and reward proxy models, and the Qwen2.5-7B-Instruct as the LLM on the server. For DP fine-tuning of the proxy models, we follow the codebase from Li et al. (2024a), training both models for 3 epochs with a batch size of 4 and a gradient accumulation step of 16. We freeze the embedding layer of the backbone and train the other parameters with a learning rate of $4e-5$. The privacy budget for fine-tuning both proxy models is set to $(8, 1e^{-5})$,

Task	Dataset	Private Train Set	Dev Set	Test Set	Sampling Synthetic Data
Medical QA	HealthCareMagic-100k	3210	112	1683	6420
Financial QA	fingpt-fiqa_qa	1693	18	1711	3386
Code Generation	opc-sft-stage2	1497	79	1449	2994

Table 7: The dataset statistics of the medical QA, financial QA and code generation task. All train set is hold by the client and is regard as the private data. The size of sampling synthetic data is two times of the size of the private train set.